

TREE 2

PROBLEM 1

QUESTION:

DS	Session	Tree 2	Question Information	Level 1	Challenge 71
Question description Given two rooted trees, your task is to find out if they are <i>isomorphic</i>, i.e., it is possible to draw them so that they look the same. Constraints • $1 \leq t \leq 1000$ • $2 \leq n \leq 10^5$ • the sum of all values of n is at most 10^5 Input The first input line has an integer t: the number of tests. Then, there are t tests described as follows: The first line has an integer n: the number of nodes in both trees. The nodes are numbered $1, 2, \dots, n$, and node 1 is the root. Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree. Output For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.					
▼ Logical Test Cases Test Case 1 INPUT (STDIN) <pre>2 3 1 2 2 3 1 2 1 3 3 1 2 2 3 1 3 3 2</pre> EXPECTED OUTPUT <pre>NO YES</pre> Test Case 2 INPUT (STDIN) <pre>2 3 1 2 2 3 1 3 3 3 3 1 2 2 3 1 3 3 3</pre> EXPECTED OUTPUT <pre>NO NO</pre>					
▼ Mandatory Test Cases Test Case 1 KEYWORD <pre>while(t--)</pre>					
▼ Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY <pre>2</pre> Test Case 2 TOKEN COUNT <pre>480</pre> Test Case 3 NLOC <pre>65</pre>					

SOURCE CODE:

```

#include <stdio.h>
#include <stdlib.h>

#define MAX_NODES 100005
#define HASH_MOD 1000000007ULL
#define HASH_MULT 1000003ULL

typedef struct AdjNode {
    int target;
    struct AdjNode* next;
} AdjNode;

AdjNode* graph1[MAX_NODES];
AdjNode* graph2[MAX_NODES];

void link_vertices(AdjNode** head_arr, int u, int v) {
    AdjNode* n1 = (AdjNode*)malloc(sizeof(AdjNode));
    n1->target = v;
    n1->next = head_arr[u];
    head_arr[u] = n1;

    AdjNode* n2 = (AdjNode*)malloc(sizeof(AdjNode));
    n2->target = u;
    n2->next = head_arr[v];
    head_arr[v] = n2;
}

int cmp_hashes(const void* a, const void* b) {
    unsigned long long ha = *(const unsigned long long*)a;
    unsigned long long hb = *(const unsigned long long*)b;
    return (ha > hb) ? 1 : ((ha < hb) ? -1 : 0);
}

unsigned long long compute_tree_hash(AdjNode** head_arr, int cur, int parent) {
    unsigned long long child_sigs[128];
    int child_count = 0;

    for (AdjNode* edge = head_arr[cur]; edge != NULL; edge = edge->next) {
        if (edge->target != parent) {
            child_sigs[child_count++] = compute_tree_hash(head_arr, edge->target, cur);
        }
    }

    qsort(child_sigs, child_count, sizeof(unsigned long long), cmp_hashes);
    unsigned long long signature = 1999ULL;
    for (int i = 0; i < child_count; i++) {
        signature = (signature * HASH_MULT + child_sigs[i]) % HASH_MOD;
    }
    return (signature ^ (signature >> 16)) * 2654435761ULL;
}

int main() {
    int test_cases;
    if (scanf("%d", &test_cases) != 1) return 0;

    while (test_cases--) {
        int n;
        if (scanf("%d", &n) != 1) break;

        for (int i = 1; i <= n; i++) {
            graph1[i] = NULL;
            graph2[i] = NULL;
        }

        for (int i = 0; i < n - 1; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            link_vertices(graph1, u, v);
        }
        for (int i = 0; i < n - 1; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            link_vertices(graph2, u, v);
        }

        unsigned long long sig1 = compute_tree_hash(graph1, 1, 0);
        unsigned long long sig2 = compute_tree_hash(graph2, 1, 0);

        if (sig1 == sig2) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}

```

OUTPUT:

```
1
3
1 2
1 3
1 2
2 3
NO
```

```
3
1 2
2 3
1 3
3 2
YES
```